Document number: P0940R3
Date: 2019-10-07 (Belfast)
Project: Programming Language C++, SG1
Authors: Michael Wong, Olivier Giroux
Email: michael@codeplay.com, ogiroux@gmail.com
Reply to: michael@codeplay.com

Concurrency TS is growing: Concurrent Utilities and Data Structures

Change log

R3 - removed moving thread clause 32, added origin of clauses and start new section on Concurrency TS3, moved dfiber_context to clause 17 where coroutines are
R2 - adds Clause 33.7 for Cooperative User-Mode Threads such as fiber_context
R1 - removed Shared_ptr based on JAX discussion
- Changed Contention to Synchronization
- Import in thread library which has futures to replace the original futures section
- Rebase this to latest Draft N4741

R0 - initial proposal

Introduction

This is a proposal for a draft new section to the C++ Standard to support SG1 Concurrency features. We foresee a number of upcoming features for inclusion. We also foresee some features in existing sections that are concurrency related that is worth moving into this new section.

There is no wording yet until we agree on the structure of the reorganization.

Concurrency TS2

A large number of Concurrency features are coming for C++20. This is because Concurrency TS1 was not added to C++17. However some of the features in it has changed. There are also many new features aiming for Concurrency TS2. Let us recap.

Concurrency TS1 was published in Jan 19, 2016[P0159] but still too late for C++17. It contains

- atomic_shared_ptr and atomic_weak_ptr class templates
- Latches and barriers
- Improvements to std::future<T> and Related APIs

Since its publication and through usage feedback, several of these facilities have been rethought. In a recent SG1 meeting in Toronto, Atomic_shared_ptr is now atomic<shared<ptr>>. Latches and barriers is undergoing a partial redesign to split the arrive/wait facilities. Even futures is being redesigned to serve the needs of executors, TLS, and other facilities better.

Concurrency TS2 is an ongoing WIP but should contain the following which has been making its way through WG21/SG1:

- Executors that links concurrency and parallelism constructs with different execution resources. There is a possibility that this may split off into its own TS.
- Data structures such as Concurrent queues, counters, Synchronized<T>, Atomic_ref<T>
- Several synchronization primitives for locked-free programming on concurrent data structures. These are cell, hazard ptr and RCU. These extends the existing shared_ptr and the proposed atomic_shared_ptr which all have safe reclamation facilities. As such we also propose moving shared_ptr and atomic<shared<ptr>> to this new location. We suspect this part may be controversial, so would ask for discussion on this topic.
- Asymmetric fences
-

Given the proliferation of these and other facilities, as Concurrency TS editor, and before we move sections and inject new wordings, we propose the following new chapter to handle these concurrency utilities for Concurrency TS2 and TS1.

At this point, there is no plan to change or update Concurrency TS1. However, not all may agree with that. We would also invite a discussion on this in the upcoming meeting.

Organization Proposal

We rebase the discussion to the latest Draft N4830. From JAX review there was consensu to continue in this direction, but also

- Keep clause 32 thread support where it is
- Keep shared_ptr where it currently is, but move atomic<shared<ptr>> from Clause 20.
- Add Fiber_context to Clause 17 coroutines

Current and Proposed future Structure

Current Draft N4830	**Future C++ Standard (Bold are new sections)**
● 32 Thread Support ○ 32.1 General ○ 32.2 Requirements ○ 32.3 Stop tokens ○ 32.4 Threads ○ 32.5 Mutual exclusion ○ 32.6 Condition variables ○ 32.7 Semaphore ■ 32.7.1 Header <semaphore> synopsis ■ Class template counting _semaphore ○ 32.8 Coordination Types ■ 32.8.1 Latches ■ 32.8.1 Header <latch> synopsis ■ 32.8.3 Class latch	

■ 32.8.4 Barriers ○ 32.9 Futures	
●	● **33: Concurrency Utilities Library** ○ **33.1 General Concepts** ■ **33.1.1 Thread Support** ■ **33.1.2 Executor Support**
	● **33.3 Executor Support** ○ **33.3.1 Executors in-depth**
● 31 Atomic operations library ○ 31.1 General ○ 31.2 Header <atomic> synopsis ○ 31.3 Type Aliases ○ 31.4 Order and consistency ○ 31.5 Lock-free property ○ 31.6 Waiting and notifying ○ 31.7 Class template atomic_ref ■ 31.7.1 Operations ■ 31.7.2 Specializations for integral types ■ 31.7.3 Specializations for floating-point types ■ 31.7.4 Partial specialization for pointers ■ 31.75 Member operators common to integers and pointers to objects ○ 31.8 Class template atomic ○ 31.9 Non-member functions ○ 31.10 Flag type and operations ○ 31.11 fences	● **33.4 Data structures** ○ **33.5.1 Concurrent queue** ○ **33.5.2 Concurrent counters** ○ **33.5.3 Synchronized<T>** ○ 33.5.4 Atomic_ref<T>

● 20.11 Smart Pointers ○ 20.11.8 Atomic specializations for smart pointers ■ 20.11.8.1 Atomic specialization for shared_ptr ■ 20.11.8.2 Atomic specialization for weak_ptr	● **33.5 Safe Reclamation** ○ 33.6.1 Atomic specializations for smart pointers ■ 33.6.1.1 Atomic specialization for shared_ptr ■ 33.6.1.2 Atomic specialization for weak_ptr ○ **33.6.2 Latest (previously Snapshot/Cell)** ○ **33.6.3 RCU** ○ **33.6.4 Hazard Pointers**
● 17.12 Coroutines ○ 17.12.1 Header <coroutines> ○ 17.12.2 Coroutine traits ○ 17.12.3 Class template coroutine_handle ○ 17.12.4 No-op coroutines ○ 17.12.5 Trivial awaitables	● **17.14 Cooperative User-Mode Threads** ○ **17.14.1 Header <fiber_context> synopsis** ○ **...**

The reason I am interested in moving atomic<shared<ptr>> into the section on concurrency in some order with Safe Reclamation is that they are actually shared concurrency structures. Shared_ptr exists where it does (Clause 23.11 Smart Pointer) because at the time, it was delivered with the Boost Smart pointer as a package. At the JAX meeting, there was consensus to not include it here. In this paper [P0233], the authors illustrate in the table in Section 7 a comparison of the capabilities between the various facilities for Reclamation. Reference Counting is the implementation behind shared_ptr and Split reference Counting (or Reference Counting with DCAS) is the implementation behind atomic_shared_ptr. These have many capabilities similar to Hazard Pointers, Cell and RCU differing only in the performance and lock-free implications.

We would ask SG1 to give guidance on this structure reorganization at the next meeting.

Acknowledgement

The author wishes to thank Maged Michael and Paul Mckenney for the comparison and early feedback. We thank SG1 for feedback.

References

[P0159] Programming Languages — Technical Specification for C++ Extensions for Concurrency
http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0159r0.html
[P0233] Hazard Pointers: Safe Reclamation for Optimistic Concurrency
http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0233r6.pdf